MinorFlow: A gem5 Pipeline Visualizer for Computer Architecture Education

Manuel Nieto[1][0009−0007−3893−5628], Francisco Cortez Casini[1][0009−0008−0892−0545], María Delfina Vélez Ibarra[1][0000−0001−6768−5313], and Gonzalo Tomás Vodanovic[1,2][0009−0001−9650−5446]

[1] Facultad de Matemática, Astronomía, Física y Computación, Universidad Nacional de Córdoba, Córdoba, Argentina
[2] Facultad Regional Villa María, Universidad Tecnológica Nacional, Villa María, Argentina
manuel.nieto@mi.unc.edu.ar

Abstract. Computer architecture is a foundational component of High-Performance Computing education, reconciling abstract algorithms with underlying hardware constraints. However, a significant pedagogical gap exists between the simplified textbook pipelines taught in undergraduate courses and the complex realities of modern processors. Cycle-level simulators, like gem5, partially bridge this gap by exposing realistic microarchitectural behavior. gem5's default text-based statistical outputs fail to convey the temporal dynamics of pipeline execution, leaving students unable to intuitively grasp stalls, cache misses, or branch mispredictions. In order to address this, we introduce MinorFlow, a web-based visualization tool tailored for gem5's MinorCPU model. MinorFlow transforms raw MinorTrace outputs into a multi-cycle pipeline diagram. The visual language follows the row-per-instruction, column-per-cycle convention familiar to students of computer architecture, and exposes events that remain hidden in aggregate statistics: fetch request and response separation, inter-stage buffer stalls, scoreboard decisions, load-store queue activity, and branch misprediction flushes. Through functional validation and targeted case studies, we show how the tool exposes microarchitectural causal relationships and makes performance bottlenecks visually identifiable. MinorFlow lowers the entry barrier to architectural analysis and offers a practical contribution to undergraduate education across the gem5 user community.

Keywords: Pipeline visualisation · gem5 · Computer architecture education · RISC-V

1 Introduction

High-Performance Computing (HPC) has become an important pillar of scientific and technological progress. From climate modeling and genomic sequencing to large-scale machine learning workloads, modern scientific research is fundamentally dependent on the ability to run computationally intensive applications on

parallel, high-throughput hardware [5]. Today, HPC infrastructure is expanding across Latin America, supported by collaborative networks among national research and education organizations that aim to consolidate a regional ecosystem of scientific computing [9]. Consequently, training a new generation of professionals capable of understanding and managing these complex architectures becomes increasingly urgent.

Computer architecture education plays a central role in this preparation. A student who comprehends the basic instruction cycle as well as the effects of cache behavior, branch prediction, and memory latency is far better equipped to write efficient software, tune simulations, or contribute to hardware design. In Latin America, the dominant pedagogical reference for this subject is Computer Organization and Design: The Hardware/Software Interface by Patterson and Hennessy [14], a book that has become the standard textbook in a large number of undergraduate programs across the region.

The pipeline model presented in that text is deliberately simplified for pedagogical purposes, featuring a clean, five-stage in-order design. While effective for introducing foundational concepts, this abstraction creates a structural gap when compared to the hardware students later encounter in research or industry. Because modern processors dynamically hide latency, aggressively manage load-store queues, and exploit instruction-level parallelism in ways the basic model does not capture, the conceptual distance between the classroom pipeline and real-world architectures becomes not just quantitative but qualitative.

To bridge this conceptual gap, educators often turn to cycle-level simulation as a practical way to expose students to realistic hardware dynamics. In this context, the gem5 simulator [4] has become a prominent tool, providing a level of microarchitectural detail that textbook abstractions cannot achieve. Its detailed CPU models offer an environment that is significantly richer than basic classroom examples yet remains tractable for undergraduate analysis. Specifically, its high-level, Python-based configuration interface allows students to easily reconfigure the underlying hardware components and contrast the behavior of alternative microarchitectures within a single experimental framework. However, the standard educational practice often reduces this tool to simply running a simulation and inspecting the aggregate metrics it reports at the end of each run. Relying entirely on this flat text summary (typically the `stats.txt` file, listing instructions-per-cycle IPC, cache hit rates, and cycle counts) renders the exercise fundamentally non-explanatory. While a student can easily observe that one configuration outperforms another, these final statistics do not convey the underlying reasons.

Visualization has long been recognized as a powerful aid in computer architecture education, helping students build viable mental models of the hardware that purely textual or mathematical descriptions struggle to convey [19,18]. Within this broad area, the specific challenge of making pipeline behavior visible is conventionally addressed by a multi-cycle pipeline diagram: a grid in which each row represents an instruction and each column denotes a clock cycle, with cells indicating the pipeline stage occupied at any given time. This representation, used

as the standard pedagogical device for pipelining in widely adopted textbooks such as [14], makes the temporal dynamics of execution directly visible, turning hazards, stalls, and forwarding into patterns that can be seen and reasoned about. On paper, however, the format only scales to short instruction sequences and short, idealized latencies.

When students then transition to cycle-level simulators like gem5, they lose even this limited visual scaffolding. The simulator's output is reduced to flat tables of cycle counts and rates, and the per-cycle behavior that the textbook diagrams made visible disappears entirely from view. The challenge, therefore, is not just simulating realistic hardware, but making its complex internal dynamics as visible and intuitive as the introductory models, at the scale that real programs actually require.

Addressing this challenge, we present MinorFlow, a web-based visualization tool that parses the MinorTrace output produced by gem5's MinorCPU model and renders it as a multi-cycle pipeline diagram directly in the browser, requiring no installation, no compilation, and no dependencies beyond a web browser. The visual language is deliberately aligned with the diagrams already familiar to students from [14], so that the transition from the textbook pipeline to a realistic in-order core is a refinement of an existing mental model rather than a replacement of it. Beyond the basic stage grid, the tool exposes the per-instruction events behind the aggregate counters of `stats.txt`: the separation between fetch request and fetch response, inter-stage buffer occupancy, load-store queue backpressure, branch misprediction flushes, and cache hit and miss events.

This combination of features also shapes how the tool is used in practice. The lightweight nature of the workflow turns MinorFlow into an instrument for experimentation, allowing students to modify a program, re-run gem5, and immediately observe the effect on the pipeline diagram, closing a loop of hypothesis, experiment, and observation. Because both gem5 and the viewer run comfortably on commodity hardware, this loop is accessible without the FPGA boards, HDL toolchains, or dedicated lab infrastructure that hardware-based pipeline experimentation traditionally requires.

The remainder of this paper is organized as follows. Section 2 reviews related visualization tools and their limitations. Section 3 describes the gem5 configuration used to generate traces. Section 4 presents the architecture and implementation of the visualization tool. Section 5 reports functional validation through microbenchmarks and cross-validation against gem5's built-in tools. Section 6 discusses pedagogical applications and case studies. Section 7 concludes with directions for future work.

2 Related Work

There are several tools that help students visualize the evolution of a pipeline during the execution of a program. Some can process instructions traces generated by external simulators such as gem5 [4], while others use an interactive datapath diagram to illustrate how individual microarchitectural components

behave during simulation. The following section reviews several of these visualizers, categorizing them into three distinct groups, all of which remain insufficient for the problem addressed in this paper.

The first group consists of the official gem5 tools. This includes minorview.py, which is the only official tool that processes MinorTrace traces generated by the MinorCPU. However, minorview's implementation relies on deprecated GUI (GTK/Tk) libraries used by Python 2, making it difficult to use on modern systems. Furthermore, it is limited to a single-cycle state view, lacking the temporal, multi-cycle pipeline perspective required to analyze the instruction flow of a program. Gem5 also includes o3-pipeview.py, which generates a multi-cycle, text-based diagram for the out-of-order O3CPU model [3] but lacks the capacity to process MinorTrace traces, and is limited by its static, text-based format, which struggles to intuitively convey complex temporal dynamics and datapath behavior at scale.

The second category comprises visualizers that display a multi-cycle pipeline stages diagram. The *de facto* standard is Konata [16], an interactive GUI-based tool, which offers pan-and-zoom capabilities, and is also capable of indicating forwarding between stages, pipeline stalls (bubbles), and several other complex microarchitectural behaviors present in modern cores. It features 47 public forks and has been adopted by projects such as RSD [13] and NaxRiscv [17]. However, all Konata parsers exclusively consume O3PipeView traces or the native Kanata format. After reviewing the source code and identifiable forks, none were found to support MinorTrace.

The third category consists of educational simulators focusing on the core hardware itself, with their own pipelines: Ripes [15] a graphical processor simulator and assembly editor for RISC-V, WebRISC-V [7,8] a web-based simulator, BRISC-V [2], the VRV system simulator [12], the web-based simulator for superscalar RISC-V processors [11], the FREESS simulator[6], and SonicRV [10]. All of these are proprietary RISC-V simulators featuring didactic 5-stage Patterson/Hennessy-style pipelines that do not process traces from an external simulator.

In summary, none of the existing tools simultaneously supports gem5's MinorTrace input, operates without installation in a standard web browser, and renders a multi-cycle-per-instruction diagram of the kind introduced in [14]. The tool presented in this paper is designed to fill precisely that gap.

3 Background

3.1 Gem5 Simulator and the MinorCPU Model

The gem5 simulator is a modular, event-driven platform for computer architecture research and education [4]. It supports full-system and syscall-emulation execution modes and provides cycle-level models of several CPU microarchitectures, ranging from simple in-order pipelines to complex out-of-order processors. Its widespread adoption in both academic and industrial settings makes it a natural choice for undergraduate courses that aim to go beyond analytical models.

With this tool, students can run real programs, observe realistic performance metrics, and experiment with microarchitectural parameters without access to physical hardware. gem5 is ISA-agnostic at the level of its CPU models. The same MinorCPU or O3CPU implementation can be instantiated for RISC-V, ARM, x86, and other supported architectures, which makes any tool built on top of its trace output equally applicable across ISAs.

Among the CPU models offered by gem5, MinorFlow targets the MinorCPU, which implements a four-stage in-order pipeline (Fetch1, Fetch2, Decode, and Execute) with configurable cache hierarchies, a Load-Store Queue (LSQ), a scoreboard-based issue mechanism, and support for dynamic branch prediction. The model is highly parameterizable: the widths of inter-stage buffers, the latencies and issue rates of each functional unit, the cache sizes and associativities, and the branch predictor structure can all be adjusted independently. Crucially for our purposes, MinorCPU emits a detailed per-instruction trace at runtime (known as MinorTrace) which records the pipeline stage occupied by each instruction at every clock cycle, along with auxiliary events such as cache requests, buffer stalls, scoreboard decisions, and branch outcomes. This trace is the primary input consumed by MinorFlow.

The choice of MinorCPU as the target model for MinorFlow reflects its pedagogical position rather than any limitation of the visualization approach itself. MinorCPU occupies a productive middle ground between gem5's simpler one-stage models and its full out-of-order O3CPU. It is substantially richer than the five-stage didactic pipeline in [14], while remaining tractable enough for undergraduate analysis.

3.2 RISC-V Configuration as a Case Study

To exercise MinorFlow in a concrete setting, and to provide a reference configuration against which the validation experiments of Section 5 are performed, we instantiate MinorCPU with a parameter set representative of a modern single-issue in-order RISC-V core. RISC-V is a natural choice of ISA for this case study for two reasons. First, the current edition of [14] adopts RISC-V as its target architecture, so students reading this paper are likely to encounter the ISA in their coursework. Second, the open-source RISC-V ecosystem provides a number of well-documented industrial cores [20,1] that can serve as design references when choosing realistic microarchitectural parameters.

Building on those references, the configuration is a basic single-issue in-order 64-bit RISC-V core running at a base clock of 100 MHz, augmented with a small set of optimizations characteristic of modern processors in the same class while remaining conceptually simple enough to keep the focus on MinorFlow itself. We refer to this configuration throughout the paper as the Reference Core, whose overall organization is shown in Figure 1.

Instructions flow from left to right through a speculative front-end (Fetch1 and Fetch2) and an in-order back-end (Decode, Execute and Commit), with a small decoupling FIFO at the input of various stages. In the front-end, Fetch1 issues one cache-line request to the L1 instruction cache from whatever PC the PC-

select multiplexer currently presents, and forwards the returned line to Fetch2. The branch predictor, comprising a local Branch History Table (BHT), a Branch Target Buffer (BTB) and a Return Address Stack (RAS), sits at Fetch2 and is consulted on every control instruction once its bytes have been pre-decoded. When the predictor calls a branch taken, Fetch2 emits a prediction that redirects PC-select to the predicted target, the upper of the two red arrows in the figure. The lower red arrow is asserted by Branch resolution in Execute. In the back-end, Decode identifies each instruction's operation, source and destination registers, after which it is issued in program order to one of the Functional units or to the LSQ, which reaches into the L1 data cache. The Functional units span a deliberately wide range of latencies, from a single-cycle integer ALU to a long, poorly pipelined integer divide. Instructions can therefore finish executing out-of-order, but Commit retires them back in program order. Table 1 gives the full set of the main configuration parameters of the Reference Core that will matter for the tool evaluation.

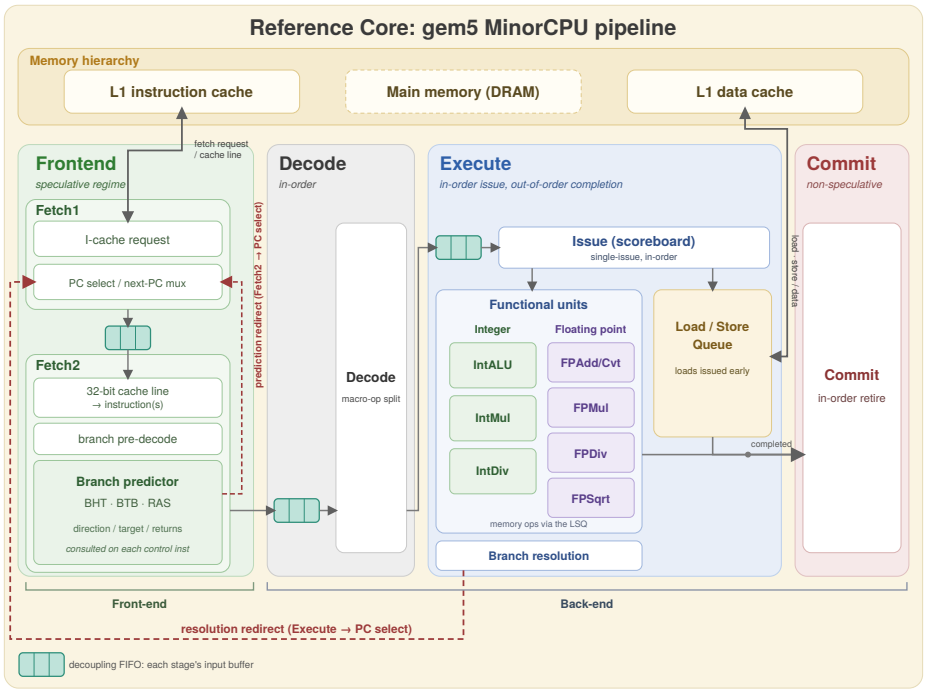

Fig. 1. Reference Core pipeline

4 Tool Description

4.1 Overview and Design Rationale

MinorFlow consumes the textual debug trace produced by gem5 and renders a timeline diagram in which each instruction occupies a row and each pipeline

stage is rendered as a horizontal bar segment colored by stage type. The result is a Gantt-style pipeline diagram that makes the propagation of each instruction through Fetch1, Fetch2, Decode, Execute, and Commit directly observable, along with the dwell times in the inter-stage buffers, the activity of the LSQ, and the cache responses.

To make that result available without specialist software, the viewer is implemented as a single self-contained HTML file with no external dependencies, runs entirely on the client side in any browser, and produces no temporary files, no caches, and no settings outside the page itself. A trace dropped onto the drop zone is parsed and rendered immediately.

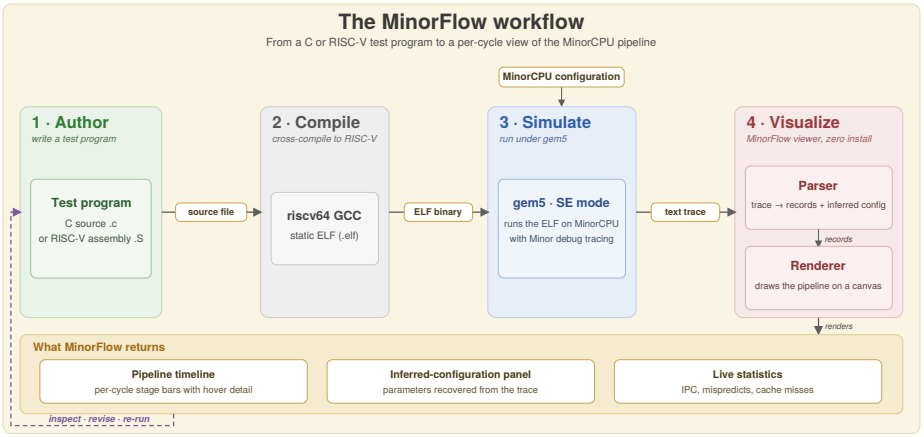

Fig. 2. The MinorFlow workflow, from a test program through gem5 simulation to the resulting timeline

Table 1. Configuration parameters of the Reference Core

Area	Parameter	Value
Pipeline	Fetch / Decode / Issue rate	1 instruction per cycle
	Commit rate	Up to 2 instructions per cycle
	Fetch1 to Fetch2 buffer	2 entries
	Decode input buffer	4 entries
	Functional unit issue queue	8 entries
Memory	Data bus width	64-bit
	L1 instruction cache	16 KiB, 4-way, 64 B line, 4 MSHRs of 8 targets, 1-cycle hit
	L1 data cache	32 KiB, 8-way, 64 B line, 8 MSHRs of 16 targets, 1-cycle hit
Branch prediction	Direction predictor (LocalBP)	1024-entry local history, 2-bit saturating counters
	Branch target buffer	256-entry, 4-way set-associative
	Return address stack	16 entries

Figure 2 shows the full workflow, from producing the trace to opening it in the viewer. A short test program is first authored in either C or RISC-V assembly. Then it is cross-compiled with riscv64 GCC into a static ELF binary, which gem5 then simulates in syscall-emulation mode against a chosen MinorCPU configuration, emitting a textual trace under the requested debug flags. Only the final step needs no toolchain at all, since the trace is just a plain-text file that is dragged onto the viewer's drop zone and rendered immediately.

The remainder of this section examines the Visualize step of Figure 2 in detail, outlining the trace format consumed by the tool and the parsing mechanisms that drive the visualization.

4.2 Trace Format

The tool consumes the standard textual output emitted by gem5 when invoked with the debug flags shown in Listing 1.1.

Listing 1.1. Trace generation command

```
build/RISCV/gem5.opt --debug-flags=Minor,MinorTrace,
    MinorTiming,CacheAll,ExecAll,Fetch,Decode,IEW,Commit,LSQ,
    Scoreboard,Writeback minorcpu_config.py code_elf
```

The trace is consumed in its raw form, with no preprocessing or auxiliary configuration file required. Traces of any length are supported, with the practical upper bound determined by the browser's available memory rather than by the tool itself.

4.3 Parser

The viewer's parser is the bridge between gem5's textual diagnostic output and the in-memory model the rendering stage operates on. Its task is to read the trace produced by a MinorCPU run and emit, for every committed instruction, the sequence of cycles at which the instruction enters and leaves each pipeline stage, along with the auxiliary information that the timeline needs to be self-explanatory.

The choice of an event-driven parser built around regular expressions follows directly from the nature of the input. MinorTrace is not a structured data format but a stream of human-readable diagnostic messages emitted by the simulator's debug infrastructure, with stable but informal formatting. In a single linear pass over the trace, the parser dispatches each line through a small set of patterns that recognized the events relevant to pipeline timing.

Correlating events across pipeline stages is straightforward because gem5's own bookkeeping carries most of the necessary information. Each instruction reported by MinorTrace is tagged with a composite identifier that names the cache line it was fetched from, a per-instruction sequence number that follows it through the back-end, and the speculation tags marking the stream and prediction it belongs to. The parser uses those identifiers as keys, so that front-end and back-end events are joined under the same instance even when they appear far apart in the trace.

4.4 Renderer

The renderer is implemented in HTML5 Canvas as three coordinated panels. A fixed left column lists each instruction's index, program counter, and mnemonic. A top ruler shows the cycle axis with tick spacing that adapts to the current zoom. The main timeline occupies the rest of the screen. The default view that opens after a trace is loaded is shown in Figure 3.

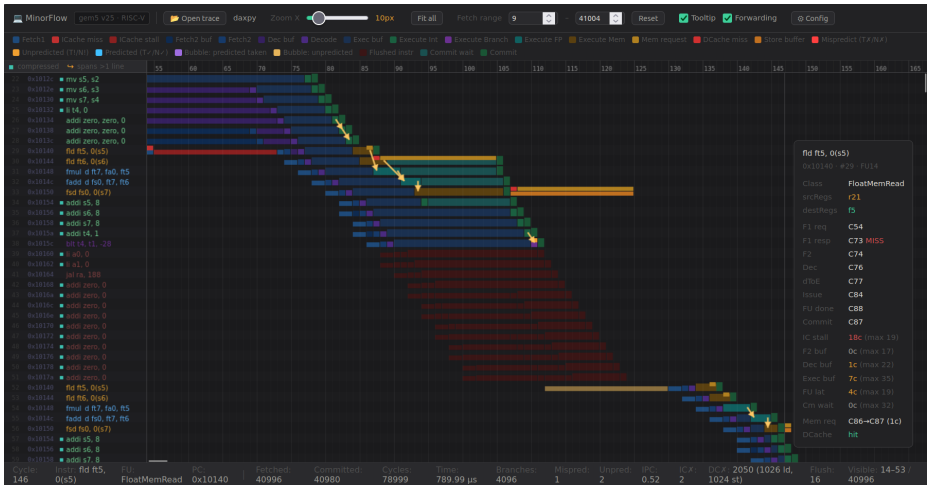

Fig. 3. Default view of a trace in MinorFlow

Rows are ordered top to bottom by commit order and within a row the colored segments record the time spent in every stage, from fetch through the execute bar (tinted by functional unit) to the final commit marker. Events outside the strict pipeline are overlaid on the same row: cache misses extend the front-end or attach a strip over the execute bar, branch-bearing rows carry a marker indicating the prediction outcome, and instructions discarded after a mispredict are drawn in red. Hovering any row brings up the per-instruction inspector visible at the right of the figure, which reports the entry cycle of every stage, the residency in every inter-stage queue and other per-instruction diagnostics.

Reading the same view at multiple scales is one of the features the viewer is built around, and it is illustrated in Figure 4. At a low zoom level the cycle axis compresses hundreds of cycles into a single screen, so the staircase shape of a program jumps out at a glance. At a high zoom level each cell is wide enough that the stage labels are drawn inside their own bars, individual cycles can be counted off against the ruler, and the forwarding overlay draws producer-to-consumer arrows over the timeline so that data dependencies become visible.

A small number of focus controls keep the viewer usable on traces that contain tens of thousands of instructions. A fetch-sequence range can be set to restrict the rows to an explicit window. The forwarding overlay and the per-instruction tooltip are each toggleable, so the timeline can be shown stripped of either layer when only the pipeline structure matters. A status bar at the bottom of the

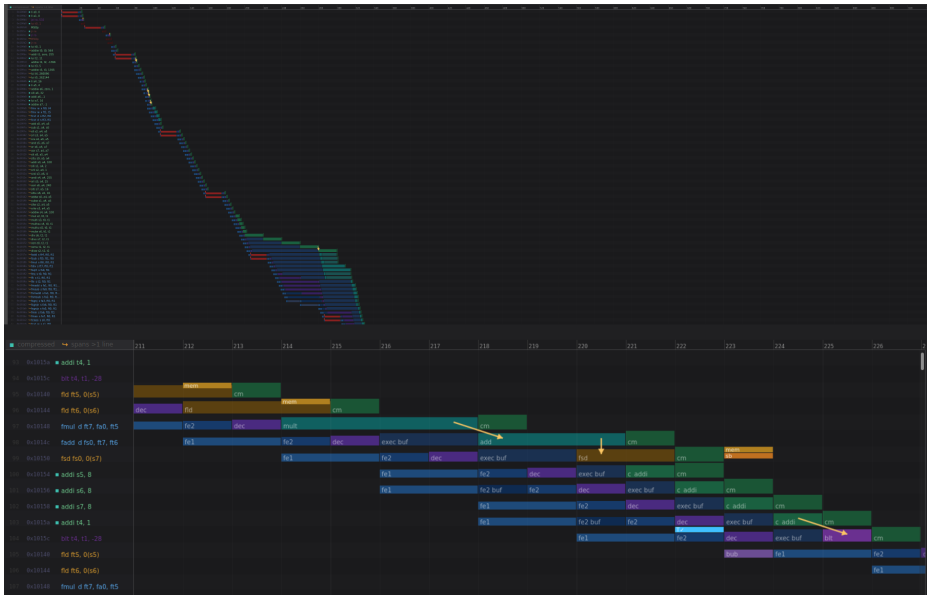

Fig. 4. Zoom options of the renderer

window updates as the visible region changes, reporting many important metrics. Finally, a config panel on the toolbar exposes the MinorCPU parameters that the parser was able to recover from the trace.

5 Tool Evaluation

5.1 Functional Validation

To validate the viewer's reported metrics, we cross-check them against gem5's own `stats.txt` and a closed-form estimate derived from the code, on the daxpy kernel $z_i = a \cdot x_i + y_i$ for $N = 4\,096$ double-precision elements. We focus on two metrics: IPC and the L1 data-cache load miss count.

In our RISC-V daxpy, the inner loop commits ten instructions per iteration (two floating-point loads, one multiply, one add, one store, three pointer increments, one counter increment and one conditional branch), so the program should commit about $4\,096 \times 10 \approx 40\,960$ instructions plus a small fixed overhead. The MinorCPU pipeline issues at most one instruction per cycle, and load-use and floating-point dependency stalls roughly double the per-iteration cost, giving a predicted IPC of about $10/20 = 0.5$. With a 64-byte cache line and 8-byte doubles, the two read arrays each touch $4\,096/8 = 512$ distinct lines, giving $1\,024$ compulsory load misses as a baseline. Table 2 reports the comparison and the corresponding status bar can be seen at the bottom of Figure 3.

Table 2. Cross-validation of MinorFlow against gem5 `stats.txt` and a hand estimate, on daxpy with $N = 4\,096$

Metric	Manual	`stats.txt`	MinorFlow
Committed instructions	$\sim 40\,960$	40 980	40 980
IPC	~ 0.50	0.519	0.52
D-cache load misses	$\sim 1\,024$	1 028	1 026

5.2 Illustrative Case Studies

To illustrate its analytical value, this section presents three scenarios where MinorFlow captures pipeline behaviors that are invisible in standard aggregate statistics. The visualizer explicitly renders the timeline of these events, enabling a clear understanding of exactly when and why they occur.

Case Study A: Functional Unit Latency and Pipeline Backpressure This scenario evaluates the execution of a code segment featuring a sequence of consecutive division operations. Because the MinorCPU is a strict in-order core, it is highly susceptible to structural hazards and backpressure caused by high-latency instructions. We expect the division operations to occupy the Execute stage for several cycles, generating backpressure toward the front-end of the core. As illustrated in Figure 5, the front-end accumulates instructions until the Execute, Decode, and Fetch2 queues reach their maximum hardware capacities (8, 4, and 2 instructions, respectively). This queue saturation ultimately triggers a fetch stall. MinorFlow allows users to visually track the exact cycle where the Execute buffer fills to capacity, as well as the cascading effect as the Decode buffer fills, leading to the aforementioned fetch stall at cycle 247.

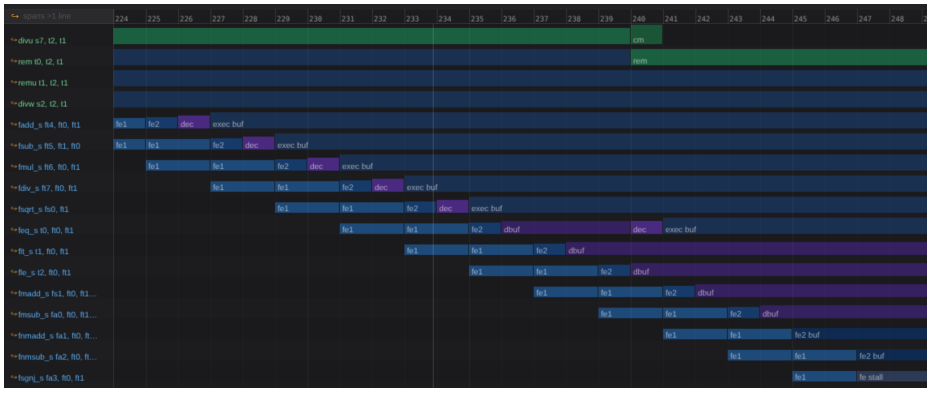

Fig. 5. Pipeline backpressure from a sequence of integer divisions (Case Study A)

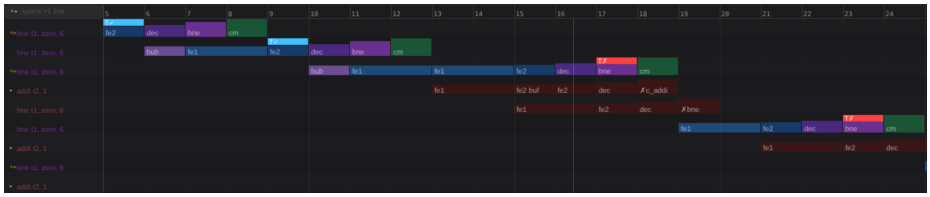

Fig. 6. Branch misprediction and pipeline flush (Case Study B)

Case Study B: Branch Mispredictions and Pipeline Flushes Speculative execution represents another critical area where aggregate metrics fall short. In this scenario, we force consecutive branch mispredictions by executing code designed to mis-train the local branch predictor. Specifically, the BHT is biased to predict a "Not Taken" outcome for branches that are actually "Taken". As shown in Figure 6, at cycle 13, the core fetches from the incorrect speculative address. It detects this error at cycle 17 when the branch instruction reaches the Execute stage. By contrast, when the prediction is correct, the flag appears in the Fetch2 stage as shown at cycle 9. Upon identifying the misprediction, the pipeline flushes all speculatively fetched instructions from the wrong execution path. MinorFlow provides a clear visual representation of this lifecycle: from the speculative fetching of the incorrect path and the exact cycle the misprediction is resolved, to the resulting pipeline flush and eventual resumption of instruction fetching from the correct target address.

Case Study C: Memory Latency and Cache Miss Stalls In this scenario, we evaluate the pipeline's behavior during an instruction cache miss. As shown in Figure 7, the first fetch stage, Fetch1, requests an instruction at cycle 7 but encounters a cache miss. This miss stops the front-end of the core, stalling the stage until cycle 25, when the instruction is finally retrieved from memory and normal fetch operations resume. MinorFlow provides an explicit, cycle-accurate visualization of this event, allowing users to measure the exact duration of the delay implied by an instruction cache miss.

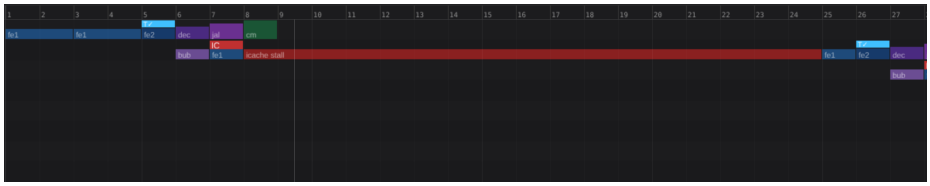

Fig. 7. I-cache miss stalling the front-end (Case Study C)

6 Pedagogical Context and Applicability

6.1 The Gap from the Student's Perspective

The motivation for MinorFlow arose directly from practical challenges encountered during two undergraduate final degree projects at FaMAF: one involving the validation of a CVA6 RISC-V core model, and another modifying MinorCPU's floating-point unit. In both instances, we hit a structural wall: neither our foundational undergraduate training nor the aggregate metrics emitted by gem5 (`stats.txt`) were sufficient to explain the complex microarchitectural behaviors we were observing. While the simulator's standard textual output accurately reported the final performance consequences, it entirely obscured the temporal events that caused them. MinorFlow was created specifically to bridge this gap from the student's perspective, providing the visual, cycle-level evidence necessary to connect abstract theoretical concepts with the realities of modern pipeline execution.

6.2 A Concrete Curricular Context: Computer Architecture Laboratory

A more structured instance of the same gap can be found in the second laboratory assignment of the undergraduate Computer Architecture course at FaMAF-UNC. In its current form, the laboratory asks students to run a set of programs under gem5 across several microarchitectural configurations, varying the size and associativity of the data cache, and switching between local and tournament branch predictors. For each configuration, the students collect the corresponding entries of `stats.txt` (cycle counts, hit and miss rates, misprediction rates) and are expected to explain why the metrics change as the configuration changes.

The pedagogical intent of the laboratory is well established and its questions are the right ones: the students are asked to reason about the microarchitectural causes of performance variation, not merely to report which configuration is faster. The difficulty lies in the evidence available to support those explanations. For example, a student who observes that a direct-mapped cache underperforms compared to an associative cache has access to the hit and miss count, but cannot directly visualize the events that produced them. With MinorFlow, the student's explanation can be grounded in what is visible rather than in what is plausible given the numbers.

6.3 Proposed Integration and Future Pedagogical Work

The integration of MinorFlow into the laboratory described above requires no change in the student-facing workflow beyond the addition of a single step. The gem5 invocation, including the debug flags listed in Listing 1.1, produces the trace as a side-effect of the existing simulation. The student then opens the viewer in a browser, drags the trace onto the drop zone, and proceeds to construct the requested explanation against the resulting diagram.

Whether MinorFlow measurably improves student comprehension is an empirical question for future work. Following established precedents [19], we plan to evaluate its pedagogical impact through controlled studies comparing student lab outcomes with and without the tool. In the meantime, MinorFlow remains available as an artifact that lowers the entry barrier to detailed microarchitectural analysis in gem5, and we hope its release encourages other groups to explore its use in their own educational settings, particularly within the region's emerging HPC communities.

7 Conclusions and Future Work

We have presented MinorFlow, a web-based visualization tool that parses MinorTrace output from gem5's MinorCPU model and renders it as a multi-cycle pipeline diagram in the browser. The tool requires no installation, and its visual language is deliberately aligned with the multi-cycle-per-instruction diagrams that students of computer architecture encounter in widely adopted textbooks. The functional validation confirms that the rendered timeline is consistent with the metrics gem5 itself reports, while the analyzed case studies illustrate how the tool exposes the cycle-level pipeline behavior.

While the current implementation is bounded by the host browser's available memory, it reliably supports traces of a few hundred thousand instructions. Beyond addressing this limitation, our next step is formally measuring the tool's pedagogical impact in classroom settings. A second, more ambitious extension would be to apply the technique to signal-level traces produced by HDL simulators such as Verilator, so that the same multi-cycle reading discipline could be brought to bear on the waveform output that students typically encounter when they move from architectural simulation to RTL. Through accessible visualization, MinorFlow aims to strengthen computer architecture education and support the growing regional HPC ecosystem.

References

1. Asanović, K., Avizienis, R., Bachrach, J., Beamer, S., Biancolin, D., Celio, C., Cook, H., Dabbelt, D., Hauser, J., Izraelevitz, A., et al.: The rocket chip generator. Technical Report UCB/EECS-2016-17, EECS Department, University of California, Berkeley (2016)
2. Bandara, S., Ehret, A., Kava, D., Kinsy, M.A.: Brisc-v: An open-source architecture design space exploration toolbox. arXiv preprint arXiv:1908.09992 (2019)
3. Bardsley, A.: O3 Model. `https://www.gem5.org/documentation/general_docs/cpu_models/O3CPU` (2026), accessed: 2026-05-22
4. Binkert, N., Beckmann, B., Black, G., Reinhardt, S.K., Saidi, A., Basu, A., Hestness, J., Hower, D.R., Krishna, T., Sardashti, S., et al.: The gem5 simulator. ACM SIGARCH computer architecture news **39**(2), 1–7 (2011)
5. Dongarra, J., Beckman, P., Moore, T., Aerts, P., Aloisio, G., Andre, J.C., Barkai, D., Berthou, J.Y., Boku, T., Braunschweig, B., et al.: The international exascale software project roadmap. The international journal of high performance computing applications **25**(1), 3–60 (2011)

6. Giorgi, R.: Freess: An educational simulator of a risc-v-inspired superscalar processor based on tomasulo's algorithm. In: Proceedings of the Workshop on Computer Architecture Education. pp. 1–8 (2025)
7. Giorgi, R., Mariotti, G.: Webrisc-v: A web-based education-oriented risc-v pipeline simulation environment. In: Proceedings of the workshop on computer architecture education. pp. 1–6 (2019)
8. Giorgi, R., Mariotti, G.: Webrisc-v: A 64-bit risc-v pipeline simulator for computer architecture classes. arXiv preprint arXiv:2504.03722 (2025)
9. Gitler, I., Gomes, A.T.A., Nesmachnow, S.: The latin american supercomputing ecosystem for science. Communications of the ACM **63**(11), 66–71 (2020)
10. Große, D., Reitinger, S., Klemmer, L.: Sonicrv: An educational platform for web-based simulation and visualization of risc-v processor architectures. IEEE Access (2026)
11. Jaros, J., Majer, M., Horky, J., Vavra, J.: Web-based simulator of superscalar risc-v processors. In: SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 1676–1684. IEEE (2024)
12. Krim, N., Porquet-Lupine, J.: Vrv: A versatile risc-v simulator for education. In: Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 2. pp. 1505–1506 (2025)
13. Mashimo, S., Fujita, A., Matsuo, R., Akaki, S., Fukuda, A., Koizumi, T., Kadomoto, J., Irie, H., Goshima, M., Inoue, K., et al.: An open source fpga-optimized out-of-order risc-v soft processor. In: 2019 International Conference on Field-Programmable Technology (ICFPT). pp. 63–71. IEEE (2019)
14. Patterson, D., Hennessy, J.: Computer Organization and Design RISC-V Edition: The Hardware Software Interface. The Morgan Kaufmann Series in Computer Architecture and Design, Morgan Kaufmann (2020), `https://books.google.com.ar/books?id=e8DvDwAAQBAJ`
15. Petersen, M.B.: Ripes: A visual computer architecture simulator. In: 2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE). pp. 1–8. IEEE (2021)
16. Shioya, D.: Konata: An instruction pipeline visualizer for Onikiri2-Kanata and Gem5-O3PipeView formats. `https://github.com/shioyadan/Konata` (2025), accessed: 2026-05-22
17. SpinalHDL: NaxRiscv core documentation. `https://github.com/SpinalHDL/naxriscv_doc` (2021), accessed: 2026-05-23
18. Yeh, T.Y., Sterner, M., Bell, C., Cowe, A.C.: Visualization with experiential learning to encourage participation and research in computer architecture. In: Proceedings of the Workshop on Computer Architecture Education. pp. 9–16 (2023)
19. Yehezkel, C., Ben-Ari, M., Dreyfus, T.: The contribution of visualization to learning computer architecture. Computer Science Education **17**(2), 117–127 (2007)
20. Zaruba, F., Benini, L.: The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-ghz 64-bit risc-v core in 22-nm fdsoi technology. IEEE Transactions on Very Large Scale Integration (VLSI) Systems **27**(11), 2629–2640 (2019)